

**LAB : Visualiser, tester et déboguer des agents avec LangGraph Studio**

## Table des matières

|                                                               |   |
|---------------------------------------------------------------|---|
| PARTIE 1 : Créer un compte et importer une clé .....          | 2 |
| PARTIE 2 : Créer un agent.....                                | 3 |
| PARTIE 2 : Fichier de configuration du graphe LangGraph ..... | 4 |
| PARTIE 2 : Installer et lancer LangGraph Studio.....          | 4 |

## PARTIE 1 : Créer un compte et importer une clé

LangGraph Studio est un environnement graphique interactif où tu peux :

- exécuter ton agent étape par étape
- voir le flux du graphe (nodes, edges)
- envoyer des messages (HumanMessage)
- inspecter les inputs/outputs de chaque node
- déboguer les outils et le RAG

1. Aller sur le site officiel

<https://smith.langchain.com/>

2. Se connecter / créer un compte

Clique sur Sign up si tu n'as pas de compte

Ou Log in si tu en as déjà un

Tu peux te connecter avec :Google, GitHub ou Email

3. Accéder aux paramètres

Une fois connecté :

Clique sur ton profil (en bas à gauche ou en haut)

Va dans Settings

4. Ouvrir la section API Keys

Dans le menu :

Clique sur : API Keys

5. Générer une nouvelle clé

Clique sur Create API Key

Donne un nom (ex : langgraph-project)

Valide

6. Ajoute la clé dans le projet :

 .env

```
1 OPENAI_API_KEY=sk-proj-wujEJzUK_v5zNBID_CDsMssqUVb9coAr3UiatFM1gBy.  
2 Groq_API_Key=gsk_k5fsftwCZtXCRse8bGvkWGdyb3FYfCyW8os2vhch9KtePoo2C  
3 TAVILY_API_KEY=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  
4 LANGSMITH_API_KEY=lsv2_pt_5ed0290dd44e42828db379e3eb88e474_4ec88ba
```

## PARTIE 2 : Créer un agent

Dans un fichier agent\_simple.py ajoute le code suivant :

```
from langchain_ollama import ChatOllama  
from langchain.messages import HumanMessage  
from langchain.tools import tool  
from langchain.agents import create_agent  
  
@tool  
def rag_search_opt(query: str) -> str:  
    """Recherche des informations dans le texte."""  
    results = "Le personnage principale est un jeune homme nommé Jack, qui découvre un ancien artefact magique. " \\  
  
    return results  
  
llm = ChatOllama(  
    model="llama3.2:latest",  
    temperature=0  
)  
  
agent = create_agent(  
    model=llm,  
    tools=[rag_search_opt],  
    system_prompt=(  
        "Tu es un assistant spécialisé dans l'analyse de texte. "  
        "Utilise l'outil rag_search_opt pour répondre aux questions en te basant sur le texte. "  
        "Réponds de manière précise et cite les informations du contexte."  
    )  
)
```

## PARTIE 2 : Fichier de configuration du graphe LangGraph

Créer un fichier langgraph.json

Il dit au système :

- où se trouve ton agent
- comment le charger
- quel environnement utiliser
- comment lancer le projet

Il sert de pont entre ton code Python et le runtime LangGraph Studio / CLI.

```
{  
  "graphs": {  
    "agent_simple": "./agent_simple.py:agent"  
  },  
  "env": ".env",  
  "source": {  
    "kind": "uv",  
    "root": "."  
  }  
}
```

## PARTIE 2 : Installer et lancer LangGraph Studio

Au terminal, lancer `uv add "langgraph-cli[inmem]"` pour installer Langgraph CLI.

Puis lancer la commande `langgraph dev` pour lancer le serveur local.

LangSmith   Studio / agent\_rag  Graph Chat Connected

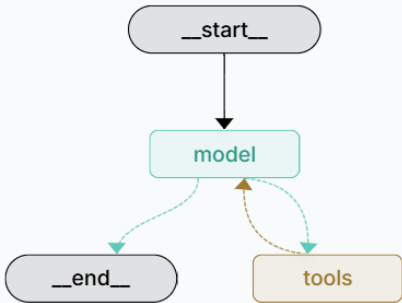
Application

- All applications 
- Search Ctrl K
- Home
- Tracing
- Monitoring
- Datasets & Experiments
- Annotation Queues
- Prompts
- Playground
- Studio**
- Deployments
- Sandboxes

Settings

**S** Personal

Memory Interrupts



```
graph TD; start([__start__]) --> model(model); model -.-> end([__end__]); model -.-> tools(tools); tools -.-> model;
```

Input   View Raw 

Messages Required 

+ Message

Manage Assistants

Submit 